

Attacco a Dizionario Parallelo per Decifratura Hash MD5

Progetto Universitario – Programmazione Parallela



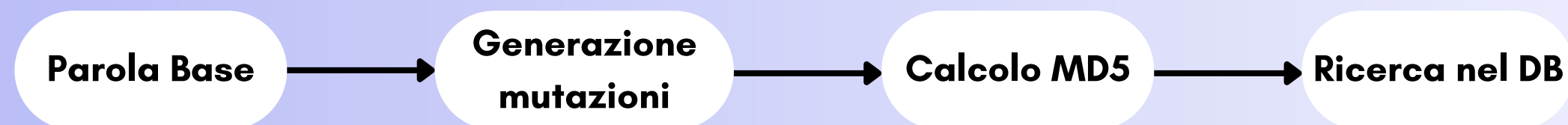
Andrea Fazzina

Scopo del progetto

Sviluppare un password cracker ad alte prestazioni in linguaggio C per decifrare hash MD5.
Il software distribuisce il carico di lavoro su architetture ibride per massimizzare il throughput crittografico.

Obiettivi del progetto

- Generazione algoritmica delle varianti (Capitalize, Uppercase, Leetspeak, Suffissi) per ogni singola parola elaborata.
- Segmentazione del dizionario dal nodo Root e invio ai worker per massimizzare le prestazioni.
- Sfruttamento simultaneo di tutti i thread logici all'interno del singolo nodo per il calcolo MD5 e per la ricerca dicotomica.



Hardware di riferimento

- Processore: AMD Ryzen AI 7 350 w/ Radeon 860M, x86_64
- Topologia Core: 8 Core Fisici / 16 Thread Logici (SMT abilitato)
- Frequenza di Clock: Base 2,00 Ghz / Max Boost 5,00 Ghz
- Cache
 - L1d: 384 KiB
 - L1i: 256 KiB
 - L2: 8 MiB
 - L3: 16 MiB
- Memoria Ram: 32 GB DDR5

Librerie utilizzate

- OpenMPI (distribuzione di rete)
- OpenMP (multi-threading locale)
- OpenSSL



Funzione crittografica

- Delegazione matematica alla libreria nativa OpenSSL.
- Estrazione dell'impronta binaria grezza (16 byte).
- Conversione forzata in formato esadecimale leggibile (32 caratteri).



Il nucleo crittografico delega l'elaborazione intensiva alla libreria nativa OpenSSL.

Poiché l'algoritmo MD5 restituisce un *digest* binario grezzo, è necessario un ciclo di conversione a esadecimale, rendendo l'hash perfettamente allineato con i record presenti nel database.

```
// Funzione crittografica di OpenSSL
MD5((unsigned const char *)str, strlen(str), digest);

// Conversione dei 16 byte grezzi in 32 caratteri esadecimali leggibili
for (int i = 0; i < 16; i++) {
    sprintf(&output_buffer[i * 2], "%02x", (unsigned int)digest[i]);
}
```

hashGen.c

Mutazioni

- Espansione del dizionario a runtime.
- Mitigazione del collo di bottiglia di I/O.
- Generazione algoritmica e indipendente di varianti per singola parola

Generare le varianti crittografiche dinamicamente in memoria centrale permette di evitare il collasso delle prestazioni legato all'I/O. Espandendo le parole a runtime tramite regole deterministiche, estendiamo esponenzialmente lo spazio di ricerca dell'attacco a costo quasi zero, mantenendo il carico di lavoro strettamente legato alla potenza della CPU (CPU-bound) anziché alla velocità del disco.

```
// 1. Prima lettera maiuscola (es. password -> Password)
void mutate_capitalize(const char *original, char *mutated)

// 2. Tutto maiuscolo (es. password -> PASSWORD)
void mutate_uppercase(const char *original, char *mutated)

// 3. Leetspeak base (es. password -> p4ssw0rd)
void mutate_leetspeak(const char *original, char *mutated)

// 4. Suffisso numerico (es. password -> password123)
void mutate_suffix(const char *original, const char *suffix, char *mutated)
```

mutazioni.c

Struttura Dati e Ricerca Dicotomica

- Allocazione della RAM in un singolo blocco contiguo
- Ordinamento alfanumerico in fase di avvio: Costo $O(N \log N)$.
- Motore di Ricerca Dicotomico: Costo $O(\log N)$.

La velocità dell'intero sistema dipende da quanto rapidamente riusciamo a confrontare ogni nuova variante con il database bersaglio. La soluzione consiste nel caricare gli hash, preventivamente ordinati, in un unico blocco di memoria contigua. In questo modo attraverso una ricerca dicotomica, vengono abbattuti drasticamente i tempi di ricerca.

```
// Allocazione contigua
db.hashes = malloc(max_hashes * sizeof(char[MD5_LEN]));

// Ordinamento (Fase di avvio)
qsort(db.hashes, db.count, sizeof(char[MD5_LEN]), compare_hashes);

// Ricerca Dicotomica (Fase di attacco)
void *result = bsearch(target_hash, db->hashes, db->count, sizeof(char[MD5_LEN]), compare_hashes);
```

hashDB.c

Direttiva OpenMP

- Parallelizzazione interna
- Spazio di memoria privato per ogni thread
- Risoluzione della Race Condition

Per preservare l'integrità dei dati e impedire fenomeni di race condition, il blocco di salvataggio è stato protetto tramite una sezione critica.

```
// Sezione critica: solo un thread alla volta può eseguire questo blocco
#pragma omp critical
{
    if (is_mutated) (*found_mutated)++;
    else (*found_original)++;

    if (*pair_count < MAX_PAIRS_PER_PROC) {
        snprintf(local_pairs[*pair_count], PAIR_STR_LEN, "%s : %s", hash_str, password);
        (*pair_count)++;
    }
}
```

logica.c

Distribuzione MPI

- Nodo Root (Rank 0) come unico lettore del file system.
- MPI_Bcast : Sincronizzazione preventiva della RAM necessaria sui worker.
- MPI_Scatter : Trasmissione dei chunk per migliorare l'efficienza in memoria.

La gestione della memoria distribuita è orchestrata dal processo Root, che analizza preventivamente la wordlist e comunica in anticipo a tutti i worker la quantità esatta di memoria da allocare.

Successivamente, la distribuzione del carico avviene frammentando il dizionario in porzioni identiche.

```
// Comunicazione della dimensione della memoria da allocare
MPI_Bcast(&chunk_size, 1, MPI_INT, 0, MPI_COMM_WORLD);

// Distribuzione frazionata del dizionario ai worker
MPI_Scatter(
    full_dictionary, chunk_size * MAX_WORD_LEN, MPI_CHAR,
    chunk, chunk_size * MAX_WORD_LEN, MPI_CHAR,
    0, MPI_COMM_WORLD
);
```

main.c

MPI + OpenMP

- Scheduling dinamico (schedule(dynamic)) per il bilanciamento del carico sui core.
- MPI_Reduce: Estrazione del Wall Time massimo (tempo del nodo più lento).
- MPI_Gather: Riversamento dei buffer locali sul Root per la ricostruzione del payload.

Nessun thread rimane inattivo qualora termini in anticipo l'elaborazione del proprio pacchetto di hash. Le password scovate e salvate nei buffer locali vengono riversate sul nodo Root.

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < chunk_size; i++) {
    process_word(...);
}

// Aggregazione statistiche
MPI_Reduce(&local_duration, &max_duration, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

// Recupero delle password decifrate
MPI_Gather(local_pairs, MAX_PAIRS_PER_PROC * PAIR_STR_LEN, MPI_CHAR,
...
global_pairs, MAX_PAIRS_PER_PROC * PAIR_STR_LEN, MPI_CHAR,
0, MPI_COMM_WORLD);
```

main.c

Test effettuati

- *Baseline* (1 MPI x 1 OMP): Esecuzione sequenziale pura.
- *Scaling Fisico* (fino a 8 Thread): Valutazione dello scaling sulle risorse fisiche.
- *Stress Test Logico* (16 Thread): Analisi del comportamento dell'architettura ibrida in saturazione di Hyper-Threading.

Processi MPI	Thread OpenMP	Core Totali (N)	Tempo di Esecuzione	Throughput (hash/s)	Speedup (S)	Efficienza (E)
1	1	1	0,296703	2359261,8	1	1
2	1	2	0,168639	4150883,99	1,759397292	0,879698646
1	2	2	0,161193	4342612,82	1,84066926	0,92033463
2	2	4	0,110065	6359885,42	2,695707082	0,673926771
2	4	8	0,109843	6372742,96	2,701155285	0,337644411
4	2	8	0,065708	10653230,2	4,515477567	0,564434696
2	8	16	0,096705	7238505,42	3,068124709	0,191757794
4	4	16	0,056492	12391088,09	5,252124195	0,328257762
8	2	16	0,046292	15121253,99	6,40937959	0,400586224

Metriche

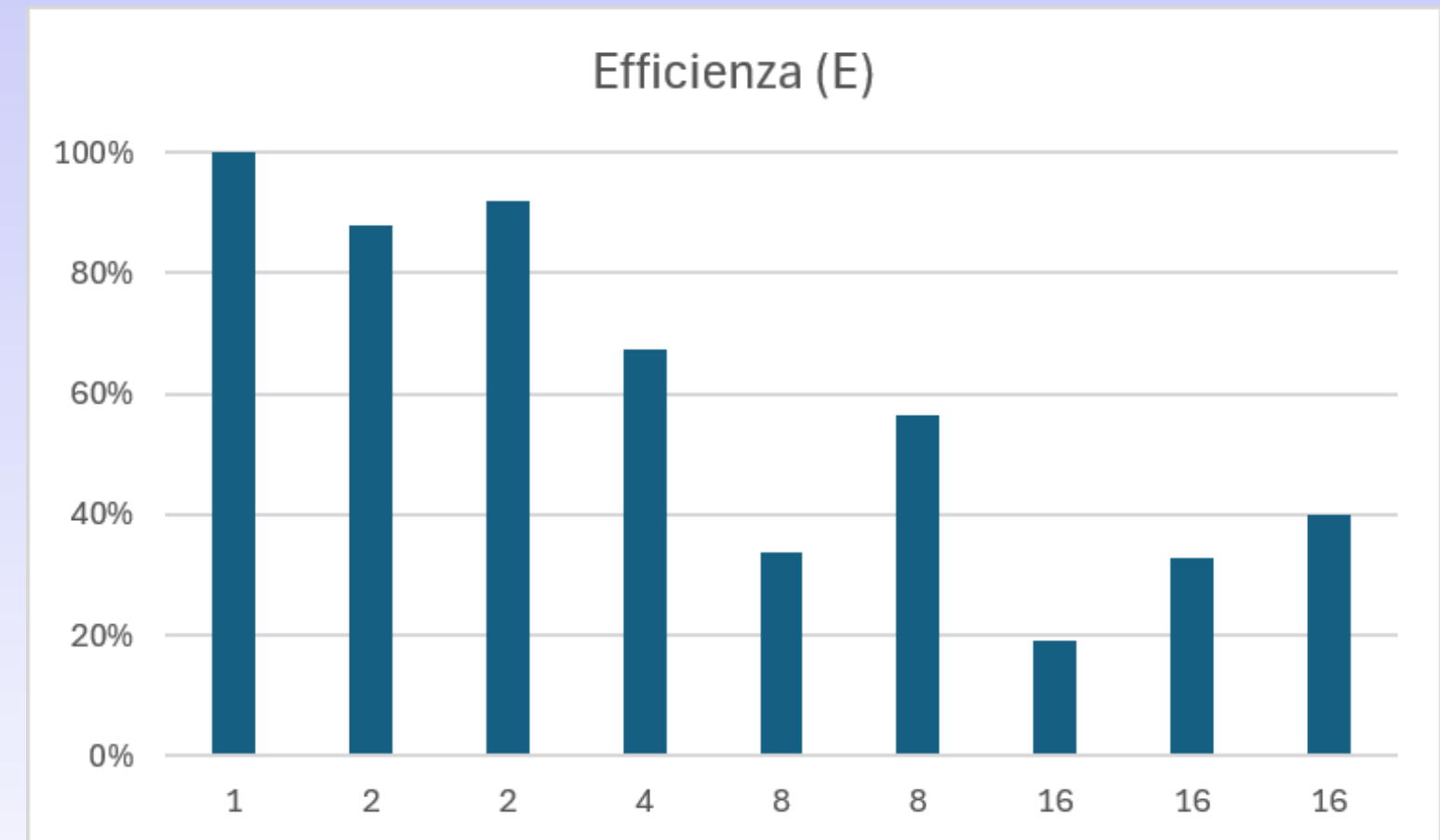
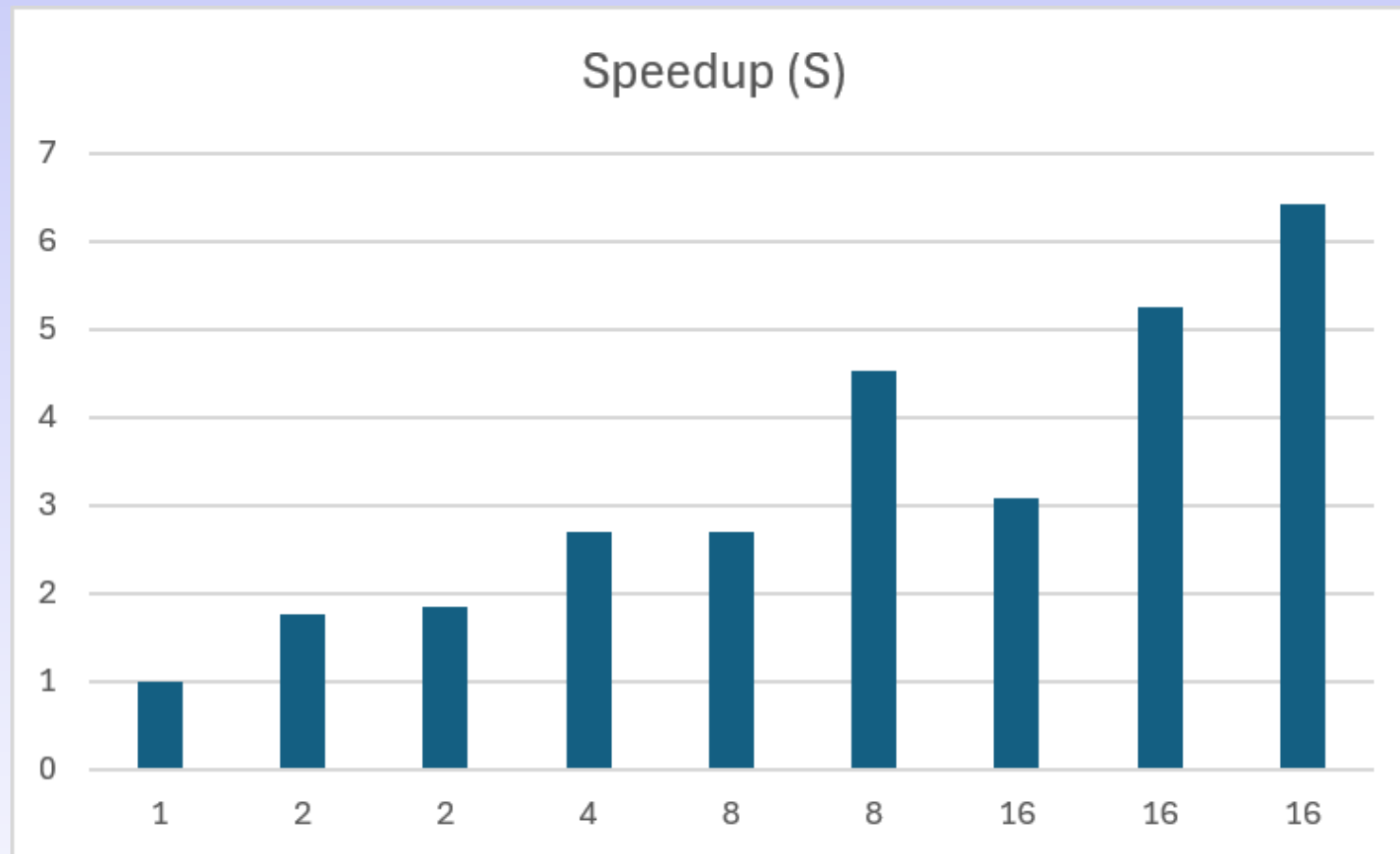
Per valutare l'architettura, abbiamo profilato i tempi calcolando Speedup ed Efficienza.

Speedup

$$S(p) = \frac{T(1)}{T(p)}$$

Efficienza

$$E(p) = \frac{S(p)}{p}$$



Analisi dell'Hyper-threading

- Saturazione delle risorse hardware (ALU).
- Superiorità dell'approccio 8 MPI x 2 OMP.
- Isolamento vs Condivisione.

I dati evidenziano come la concentrazione di troppi thread OpenMP all'interno del medesimo processo generi un collo di bottiglia. Istanziare un maggior numero di processi MPI indipendenti, limitando i worker OpenMP, riduce drasticamente le collisioni sui buffer di memoria condivisa. Questa strategia di partizionamento si è rivelata fondamentale per mitigare l'overhead e massimizzare il throughput.

Processi MPI	Thread OpenMP	Core Totali (N)	Tempo di Esecuzione	Throughput (hash/s)	Speedup (S)	Efficienza (E)
2	8	16	0,096705	7238505,42	3,068124709	0,191757794
4	4	16	0,056492	12391088,09	5,252124195	0,328257762
8	2	16	0,046292	15121253,99	6,40937959	0,400586224

Criticità di rete e Limiti Strutturali

- Collo di bottiglia I/O: il nodo Root è l'unica entità autorizzata alla lettura del disco.
- Saturazione della rete e della RAM causata dalla centralizzazione di MPI_Scatter.
- Vincolo geometrico di MPI_Scatter.

L'implementazione tramite MPI_Scatter genera due criticità architetturali.

In primo luogo, costringe un singolo nodo a caricare l'intero dizionario in RAM e a ritrasmetterlo, intasando la rete. In secondo luogo, la rigidità geometrica di MPI_Scatter impone la divisione intera delle parole per il numero di processi. Questo arrotondamento per difetto causa inevitabilmente lo "scarto" delle ultime parole del dizionario.

```
MPI_Scatter(  
    full_dictionary, chunk_size * MAX_WORD_LEN, MPI_CHAR,  
    chunk, chunk_size * MAX_WORD_LEN, MPI_CHAR,  
    0, MPI_COMM_WORLD  
);
```

main.c

```
andrea@HP-Omen-Andrea:~/codice/Prog-Parallela/progetto$ OMP_NUM_THREADS=2 mpirun -np 6 ./cracker  
Processo ROOT: Caricate 99996 parole (Limite: 100000). Chunk size = 16666 parole per processo.
```

Output

Seconda Versione

- Accesso Simultaneo (I/O Distribuito): Ogni processo apre il file in autonomia.
- Partizionamento Asimmetrico in Byte: Il carico viene diviso calcolando i byte totali del file.
- Algoritmo di Riallineamento (Look-back).

Sostituendo la distribuzione centralizzata di `MPI_Scatter` con letture indipendenti calcolate sui byte, si elimina il collo di bottiglia sul nodo Root e si ottiene una totale copertura del dizionario.

```
// RIALINEAMENTO SUI CONFINI DI MEMORIA (Tecnica Look-back)
if (rank != 0) {
    fseek(file, start_byte - 1, SEEK_SET); // Analisi del byte precedente
    char c = fgetc(file);

    // Se atterriamo a metà di una parola, la saltiamo
    // (sarà completata interamente dal rank precedente)
    if (c != '\n') {
        while ((c = fgetc(file)) != '\n' && c != EOF);
    }
}
```

```
// Salto diretto al byte di partenza assegnato
fseek(file, start_byte, SEEK_SET);
```

main_bonus.c

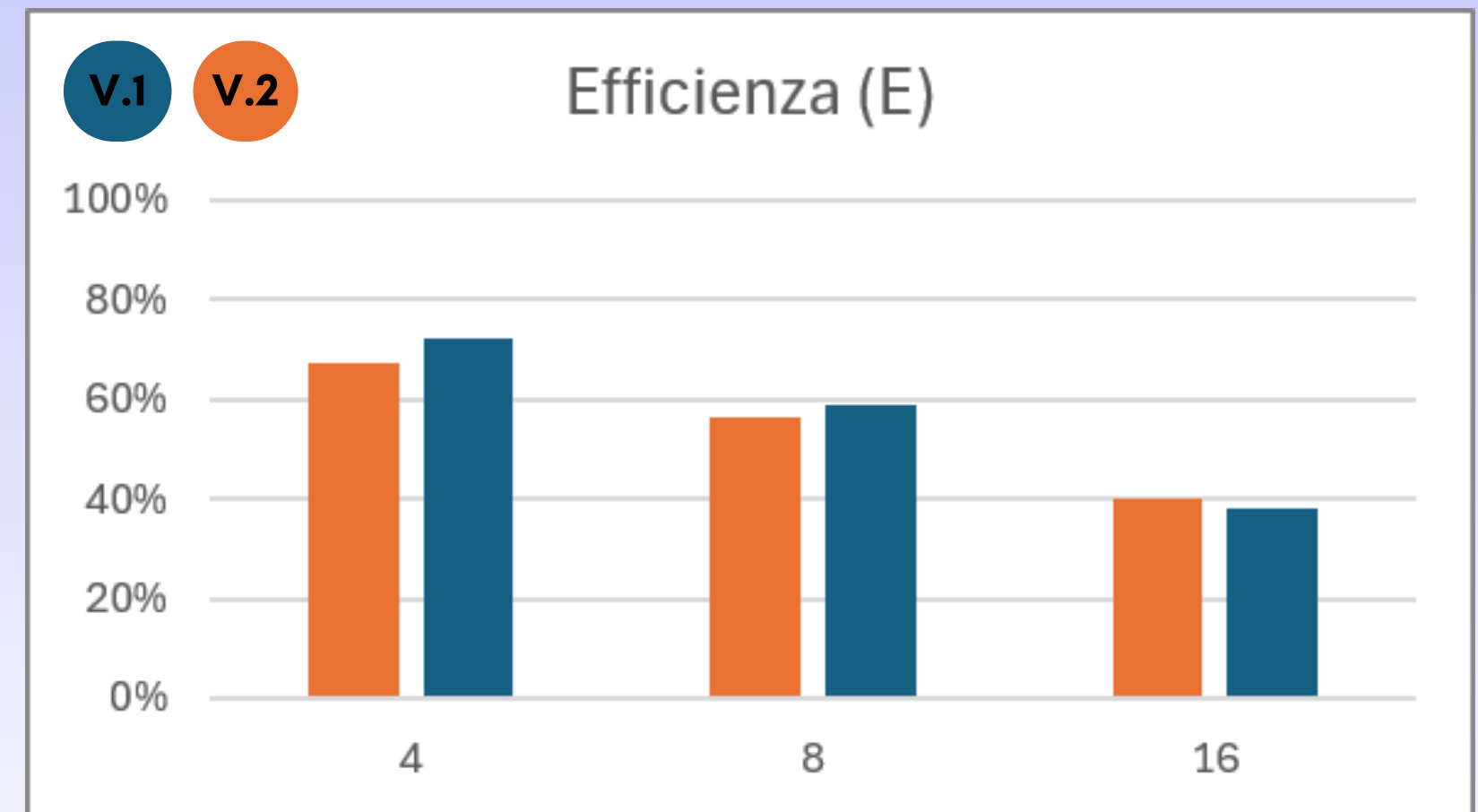
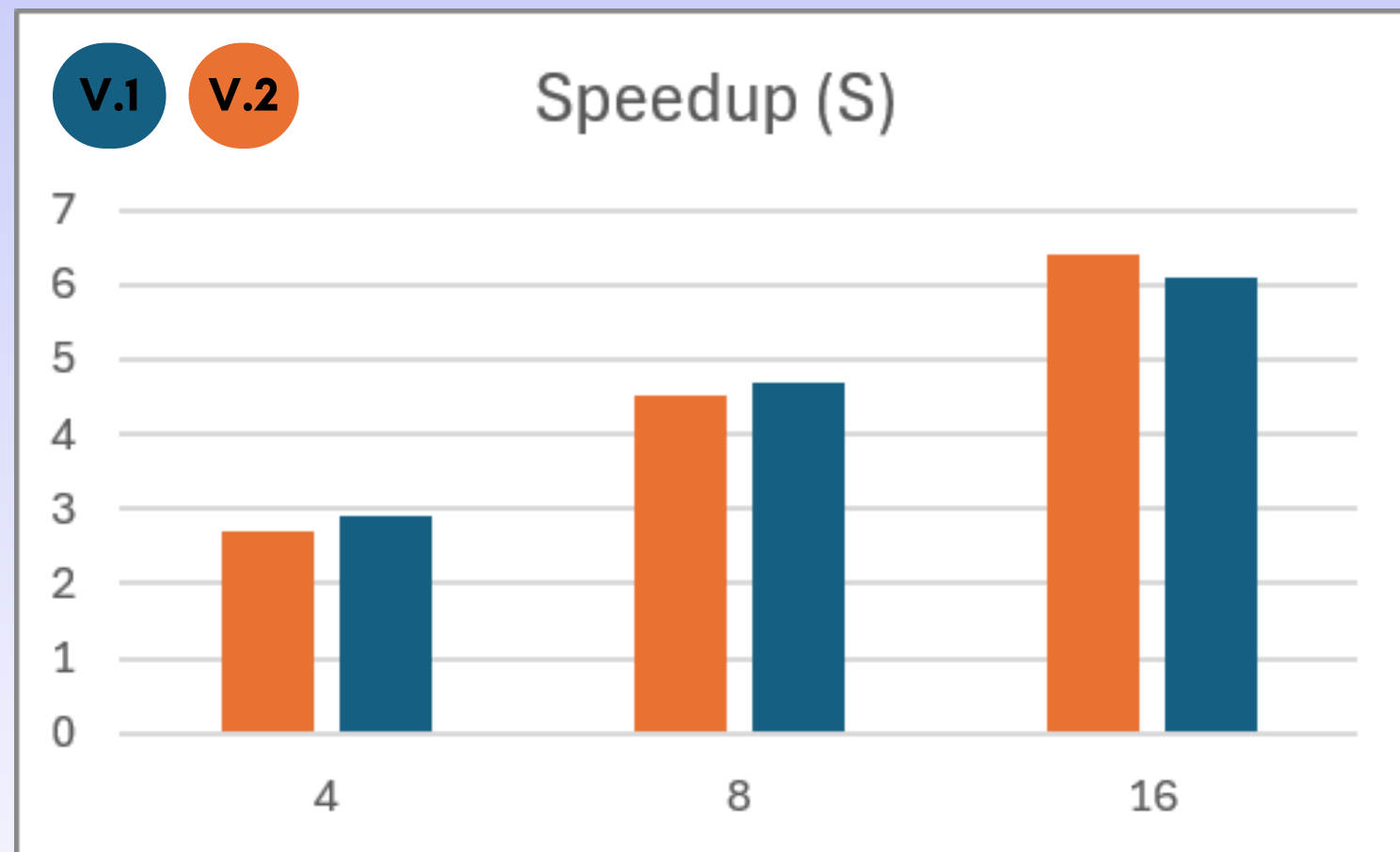
```
andrea@HP-Omen-Andrea:~/codice/Prog-Parallela/progetto$ OMP_NUM_THREADS=2 mpirun -np 6 ./cracker_bonus
Processo ROOT: Caricate 100000 parole totali (Limite: 100000). Chunk medio = 16666 parole per processo.
```

Output

Confronto Metriche

- Thread OpenMP 2 per processo per evitare colli di bottiglia locali e overhead da Hyper-Threading.
- Raddoppio progressivo dei processi distribuiti (2, 4 e 8 MPI).

Per il confronto definitivo sono state selezionate esclusivamente le configurazioni più performanti, isolando così le reali differenze di scalabilità tra la lettura centralizzata (V.1) e l'I/O distribuito (V.2).



Trade-off architetturale

- V1 (MPI_Scatter)
- V2 (I/O Parallelo)

I test finali dimostrano l'esistenza di un trade-off.

A bassa parallelizzazione, la Versione 1 risulta marginalmente più veloce grazie a un bilanciamento del carico matematicamente perfetto. Tuttavia, all'aumentare dei nodi, l'overhead di rete e la saturazione della RAM sul nodo Root ne frenano drasticamente le prestazioni.

Scalando a regime di saturazione (16 thread logici), l'I/O distribuito della Versione 2 annulla i tempi di trasmissione compensando il partizionamento in byte, superando la V1 in termini di Speedup e confermandosi la soluzione architetturale ottimale e scalabile per l'elaborazione di wordlist massive.



FINE



Andrea Fazzina